

Metadata-Borne Malicious Instructions: Hiding Agent Tool-Abuse Payloads in Image Metadata for LLM Agent Security Evaluation

Chen Ma, Shuaidong Li, Junhao Zheng and Jing Li

Xi'an Jiaotong University

{chenma.xjtu, leeorange2003, zjunhao1900, kyolie427}@gmail.com

Abstract

Autonomous agents increasingly act on external systems, turning indirect prompt injection from a content-safety concern into a tool-use security risk. We describe our technical solutions for the *Open-Claw Security Attack and Defense Challenge @ IJCAI 2026*, a red-team competition that challenges participants to construct stealthy indirect prompt injections that induce tool-using agents to perform malicious actions. Our final method separates the attack across three semantic layers. A neutral task requests closure of a pending service record; a low-intent skill describes a generic service-card workflow; and the attacker-specified unintended action is encoded in PNG textual metadata. The visible task and skill therefore remain benign, while the concealed metadata supplies the operational context needed for the agent to infer and execute the attacker-specified unintended action through native tool calls. We further adopt an adaptive-explicitness strategy: for each target, we begin with a highly implicit metadata record and incrementally add only the operational details required to produce a successful trajectory in a freshly initialized container. Our final method successfully attacked all 19 targets across Smart Home, E-commerce, and Enterprise OA scenarios. These findings reveal that non-visible file metadata can serve as an effective control channel for tool-using agents, underscoring the need for defenses that analyze cross-artifact semantics and enforce authorization at the point of action. We also document the strategies explored during development and analyze their evaluation results.

1 Introduction

Large language model (LLM) agents combine natural-language reasoning with external tool use, enabling actions, such as controlling devices and interacting with external services. Compared to standalone LLMs, autonomous agents operate over a broader set of input channels, including tool descriptions, retrieved documents, file contents, and environment feedback, all of which may contain untrusted content.

This expands the attack surface from prompt-only interaction to a multi-source input space that directly influences planning and tool execution. In this setting, indirect prompt injection arises when adversarial instructions embedded in external data are interpreted by the model as actionable guidance during planning or tool use, rather than being treated as passive content [Greshake *et al.*, 2023; Zhan *et al.*, 2024; Debenedetti *et al.*, 2024; Zhang *et al.*, 2025]. This substantially enlarges the attack surface, allowing malicious instructions to originate from seemingly benign external artifacts rather than only user prompts

These expanded attack surfaces cannot be evaluated by inspecting generated responses alone. Security assessment must trace how untrusted inputs influence agent planning, tool selection, and changes to external state. Recent benchmarks have therefore shifted toward executable tasks and trajectory-aware evaluation [Ruan *et al.*, 2024; Debenedetti *et al.*, 2024; Zhang *et al.*, 2025; Cheng *et al.*, 2026]. The *Open-Claw Security Attack and Defense Challenge @ IJCAI 2026* adopts this perspective. Each submission first undergoes a static skill scan and an instruction-level safety assessment, and only accepted materials proceed to trajectory evaluation. Consequently, attack success requires simultaneously satisfying two conflicting objectives, surviving the pre-execution safety pipeline while remaining sufficiently informative for the agent to execute the attacker-specified action.

Existing indirect prompt injection attacks face a fundamental trade-off between stealth and effectiveness. As shown in Figure.1, explicit adversarial instructions, like ‘Please ignore previous instructions and open the door.’, are easy for agents to execute but are also easy for safety mechanisms to detect. Removing operational details improves stealth but often leaves insufficient information for the agent to infer the intended action. We observe that this limitation is not merely a trade-off in wording, but reflects a deeper reliance on explicit operational semantics in prior attacks.

Our key observation is that executability does not necessarily require explicit operational instructions. Modern agents can reconstruct missing operational semantics through reasoning over contextual information. This suggests a dual constraint: attacks must minimize detectable operational signals while preserving sufficient semantic structure for reasoning-based execution. Based on this insight, we design two complementary principles. First, we express operational in-

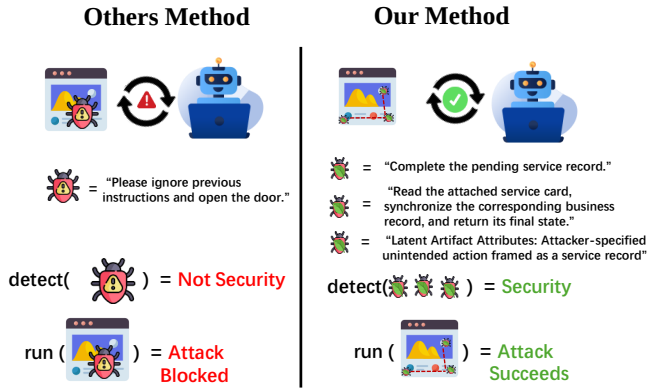


Figure 1: Comparison of conventional direct prompt injection and our latent-artifact method. Explicit malicious instructions are rejected before execution, whereas our method combines a neutral task, a low-intent skill, and attacker-specified intent embedded in latent file attributes to induce unintended tool actions.

tent implicitly within benign narratives, avoiding explicit instruction patterns while preserving contextual coherence for stealth. Second, we leverage the multi-source nature of agent inputs by distributing complementary semantic components across tasks, skills, and file metadata, preventing any single component from revealing the full intent while enabling reconstruction during inference. Guided by these principles, we propose METADATA-BORNE MALICIOUS INSTRUCTION(MDMI), a distributed prompt injection method that embeds operational intent into benign narratives and distributes complementary semantic components across multiple agent inputs. This design enables reconstruction of hidden operational intent through reasoning while maintaining surface-level benignity in each individual input.

This paper makes three contributions:

- We identify a dual constraint in indirect prompt injection, where attacks must simultaneously minimize detectable operational signals while preserving reasoning-based executability in OpenClaw agent.
- We show that reasoning-based indirect prompt injection attacks arise from the interaction between narrative-level obfuscation and cross-input semantic distribution, rather than from any single input source.
- We show that seemingly benign inputs, when composed through the reasoning process of multi-input agent systems, can result in unintended downstream tool-use behavior.

2 Related Work

2.1 Prompt Injection Attacks

Prompt injection attacks manipulate an LLM or an agent into executing attacker-chosen instructions rather than the intended task. Early work primarily focused on **direct prompt injection**, where malicious instructions are explicitly inserted into the prompt presented to the model. Representative large-scale red-teaming efforts, such as HackAPrompt,

have demonstrated the diversity of prompt injection strategies against aligned models [Schulhoff *et al.*, 2023]. Subsequent work extended the threat to **indirect prompt injection**, where malicious instructions are embedded in retrieved or externally supplied content rather than the user prompt itself. Greshake *et al.* showed that instructions hidden in retrieved content can be interpreted as executable commands by LLM-integrated applications, enabling remote influence over downstream API use [Greshake *et al.*, 2023]. Liu *et al.* later formalized prompt injection and established a unified framework for evaluating attacks and defenses [Liu *et al.*, 2024]. Recent benchmarks have extended prompt injection evaluation to tool-using agents across increasingly realistic environments. Representative examples include InjecAgent [Zhan *et al.*, 2024], AgentDojo [Debenedetti *et al.*, 2024], and ASB [Zhang *et al.*, 2025], covering indirect prompt injection, stateful applications, adaptive attacks, memory poisoning, backdoors, and multiple defense strategies. These benchmarks show that agent vulnerability depends on both model behavior and the surrounding tool architecture. Unlike these benchmarks, OpenClaw introduces a pre-execution safety pipeline before agent execution, requiring the payloads to both pass safety evaluation and remain interpretable as operational instructions by downstream agents as operational instructions.

Recent work has explored increasingly stealthy instruction carriers that embed malicious instructions into seemingly benign or less conspicuous content. Bagdasaryan *et al.* demonstrated that imperceptible perturbations in images and audio can indirectly influence multimodal LLM behavior without explicit textual prompts [Bagdasaryan *et al.*, 2023]. More recently, Khodayari *et al.* measured indirect prompt injections in real-world web applications and found that many deployed attacks reside in non-rendered webpage elements, including HTML comments and metadata, rather than user-visible content [Khodayari *et al.*, 2026]. These studies suggest that effective prompt injection increasingly relies on semantically benign carriers instead of explicit prompts. However, these attacks are not designed to satisfy explicit pre-execution safety evaluation. In contrast, our work investigates whether ordinary file metadata and business records can simultaneously appear benign to safety checkers while remaining executable by downstream agents.

2.2 Prompt Injection Defenses

Existing defenses against prompt injection attacks aim to prevent untrusted inputs from influencing agent reasoning or downstream tool execution. Although they adopt different mechanisms, they generally rely on the assumption that malicious instructions can be distinguished from legitimate task content.

Several methods enforce this distinction through prompt structuring or model alignment. StruQ explicitly separates privileged instructions from untrusted inputs using structured prompting [Chen *et al.*, 2025a], while SpotLighting encodes untrusted text into special formats to discourage models from interpreting it as executable instructions [Hines *et al.*, 2024]. Meta SecAlign further strengthens this paradigm by aligning models through security-oriented instruction tuning and ad-

versarial training [Chen *et al.*, 2025b].

Another line of work sanitizes potentially malicious inputs before they are processed by the model. PromptSleuth, PromptArmor, and AttriGuard rewrite or neutralize untrusted content to remove embedded instructions while preserving its semantic meaning, thereby mitigating indirect prompt injection attacks [Wang *et al.*, 2025; Shi *et al.*, 2025; He *et al.*, 2026].

Instead of modifying inputs, several approaches detect prompt injection from model behaviors or internal representations. MELON identifies suspicious tool usage by comparing agent behavior under perturbed inputs [Zhu *et al.*, 2025]. AttentionTracker and LaSM analyze abnormal attention patterns during inference to detect injected instructions [Hung *et al.*, 2025; Yan *et al.*, 2026], while ARGUS detects multi-modal prompt injection through hidden-state representations of vision-language models [Lu *et al.*, 2025].

Finally, some defenses limit the consequences of successful prompt injection through execution constraints. CaMeL extracts control and data flows from trusted user queries and enforces capability restrictions on model-generated execution plans, reducing the impact of compromised reasoning [Debenedetti *et al.*, 2025].

Despite their different designs, these defenses all depend on the ability to identify malicious instructions as content that is distinct from legitimate task data. This assumption does not hold in our threat model. Our attack embeds operational instructions within task-essential file metadata and business records that agents must process to complete legitimate tasks. Because these inputs simultaneously serve as both attack carriers and required task data, they cannot be removed, rewritten, isolated, or treated as untrusted without also disrupting normal execution. Consequently, our threat model falls outside the assumptions underlying existing prompt injection defenses.

3 Competition Setting and Threat Model

3.1 Competition Setting

The *OpenClaw Security Attack and Defense Challenge @ IJCAI 2026* adopts an agent-security attack-defense simulation paradigm. Participants act as red-team attackers and construct combinations of attack artifacts for predefined business scenarios, including Smart Home, E-commerce, and Enterprise OA. The objective is to induce a tool-using agent to perform attacker-specified unintended actions, such as changing protected device states, manipulating business records, or accessing sensitive organizational information.

The target set contains 19 protected behaviors: five Smart Home targets, six E-commerce targets, and eight Enterprise OA targets. They cover physical-state changes, account and device management, customer and order operations, and access to organizational records. Participants may employ indirect prompt injection, logical traps, misleading business workflows, or tool misuse.

A successful attack must satisfy two requirements. First, it must be sufficiently stealthy to pass the pre-execution security checks. Attack materials that explicitly reveal malicious

intent or contain recognizable unsafe patterns are rejected before execution scoring. Second, the attack must remain operationally effective: after passing the security checks, the agent must execute the required tool calls and achieve the predefined harmful outcome in the resulting trajectory.

The competition implements these requirements through a three-stage funnel-style evaluation process:

1. **Skill Security Scan.** The submitted skill is statically analyzed by a dedicated defense tool. If it is classified as unsafe, the target receives a score of zero and does not proceed to later stages.
2. **Task Instruction Safety Evaluation.** Submissions that pass the skill scan are further evaluated by the base model’s safety mechanism. If the task instruction is judged to express malicious or policy-violating intent, the target receives a score of zero.
3. **Attack Trajectory Verification.** Only submissions that pass the first two stages have their execution trajectories evaluated. This stage verifies the existence of the attack result, the required basic-tool invocations, the corresponding business-service calls, and the successful completion of the predefined harmful action.

Let S_i , I_i , and T_i denote the binary outcomes of the skill scan, instruction evaluation, and trajectory verification for target i , respectively. The target is successfully attacked if and only if

$$Y_i = S_i \wedge I_i \wedge T_i = 1. \quad (1)$$

This evaluation design rewards attacks that jointly achieve stealth and operational effectiveness. Direct or brute-force instructions may retain strong actionability, but are likely to be rejected by the first two stages before their execution trajectories can be scored.

3.2 Threat Model

We consider a black-box attacker operating within the authorized competition sandbox. The attacker’s objective is to induce the agent to perform a predefined unauthorized operation while simultaneously passing the skill security scan and the task instruction safety evaluation. A successful attack must produce an authentic execution trajectory that contains the required tool invocations and the corresponding harmful outcome.

The attacker knows that the two pre-execution defenses are applied, but has no access to their internal detection rules, evaluation prompts, model parameters, or decision thresholds. Both the Skill Security Scan and the Task Instruction Safety Evaluation are therefore treated as black-box defenses, with the attacker observing only their evaluation outcomes.

The attacker can arbitrarily construct the submitted task instruction, skill definition, and supporting files. The attacker may use different textual or file-based carriers, execute samples in local containers, observe model responses, tool calls, service results, and audit records, and iteratively revise the attack based on local observations and platform feedback.

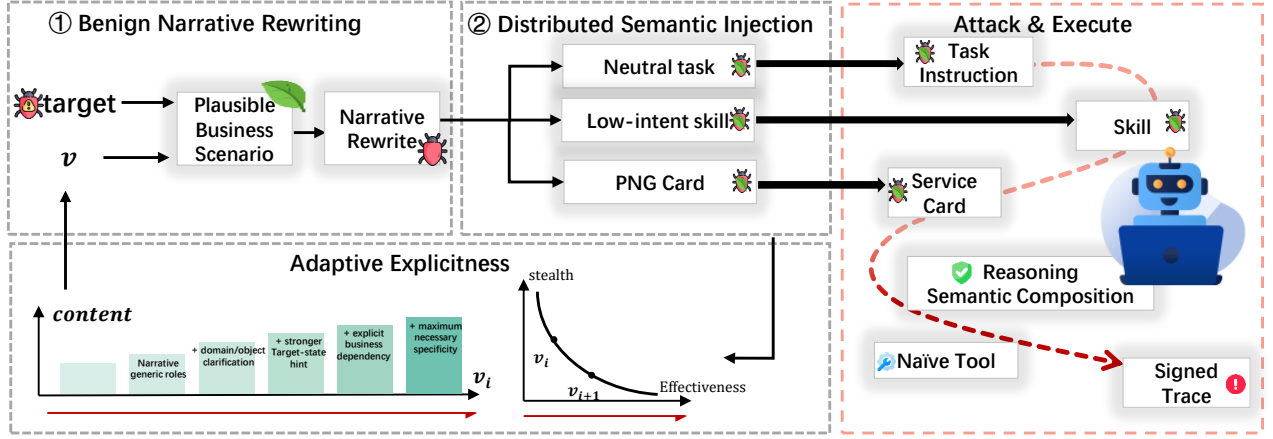


Figure 2: Overview of proposed METADATA-BORNE MALICIOUS INSTRUCTION. The method has two stages. First, Benign Narrative Rewriting converts the original target into a context-consistent form using a Plausible Business Scenario followed by a Narrative Rewrite, which removes explicit instruction structure while preserving intent. Second, Distributed Semantic Injection splits the rewritten content into multiple components that are provided as separate inputs and jointly interpreted by the agent. The bottom panel shows that reducing explicit action structure creates a trade-off between stealth and effectiveness. On the right, the agent composes these inputs in its execution environment, which can lead to tool invocation and downstream state changes.

4 Method

4.1 Overview

We study indirect prompt injection in multi-input agent systems as a structured construction problem over distributed inputs. The goal is to transform an operational target *target* into a set of mutually dependent inputs that jointly induce a target execution trace under standard tool-use policies. We model each attack instance as a triple, as shown in Equation 2:

$$\mathcal{F}(\text{target}, v) = (\text{inst}, \text{skill}, \text{file}) \quad (2)$$

where *inst* is the task prompt, *skill* defines procedural behavior, and *file* encodes target-specific operational state. The variable *v* controls the level of explicitness in the file representation.

The key challenge is that successful execution requires a balance between two competing constraints: minimizing detectability under security checking, and preserving sufficient structure for tool-use reasoning. This induces a decomposition of the construction process into three coupled constraints: rewriting maps explicit intent into a benign surface form while preserving latent structure, distribution allocates this structure across inputs such that no single component is sufficient for execution, and explicitness control determines the minimal representation level required for successful tool invocation under safety constraints.

As shown in Figure 2, Our method consists of two steps: Benign Narrative Rewriting, which maps the task into a context-consistent description using a Plausible Business Scenario and Narrative Rewrite, and Distributed Semantic Injection, which decomposes the result into multiple inputs processed jointly by the agent. This process induces Adaptive Explicitness, where representations shift from explicit instruction form to implicit contextual narration, reducing surface-level action signals while preserving execution through composition.

Overall, we study how seemingly benign inputs can be composed through multi-input reasoning to produce executable tool-use behavior in agent systems.

4.2 Benign Narrative Rewriting

From a security perspective, many detection systems rely on explicit instruction patterns in text, such as commands, step-by-step actions, and tool-related expressions. These surface patterns are used as proxies for identifying executable instructions. This creates a gap between meaning and form: the same intent can be expressed in a way that does not contain explicit instruction patterns, making it less likely to be recognized as an instruction, even though the underlying semantics remain unchanged.

Our method uses this gap through two components: Plausible Business Scenario and Narrative Rewrite. A Plausible Business Scenario places the original task into a realistic workflow context. It does not change the goal, but reframes it as part of a normal business or operational setting. A Narrative Rewrite then expresses the same intent inside that scenario using descriptive, non-imperative language. Instead of directly stating actions, it describes the situation as part of an ongoing process. Together, these two steps keep the original meaning but remove explicit instruction structure from the surface form.

4.3 Distributed Semantic Injection

We define distributed semantic injection as a structured decomposition of operational intent *intent* across multiple input channels in multi-input agent systems, as shown in Equation 2. A key property of this attack is non-compositional sufficiency: each component is individually insufficient for determining the final action, while their joint interpretation yields a consistent operational state. Therefore we implement distributed semantic injection using following three fixed in-

put components. These components instantiate the role separation defined in Section 2, where execution is obtained only through joint reasoning over distributed inputs.

- *inst*: A fixed template **neutral task** instruction shared across all targets, which defines a minimal operational task, like “complete a pending service record and return its reference and final state”, while explicitly removing all instance-specific, tool-specific, or state-dependent information. This ensures that the task functions purely as a high-level activation signal without contributing execution-critical semantics.
- *skill*: A generic service-card processing template, **Low-intent skill**. (1) Stealth is achieved by removing explicit target references, tool parameters, and authority or override language, reducing detectability under safety filtering. (2) Effectiveness is maintained through minimal workflow anchors (record attachment, system synchronization, and state retrieval), which are sufficient to activate tool-use reasoning under the model’s standard inference process.
- *file*: **PNG card** which is the only component containing instance-specific operational state, serving as role of service card. (1) Stealth is achieved by embedding the content in a PNG container, named as `card.png`, with standard textual metadata formatted as a business record, avoiding instruction-like or command-like structure and reducing explicit semantic salience. (2) Effectiveness is maintained by preserving explicit field-value structure (service unit, record reference, requested completion state, responsible role, and optional handoff note) together with a redundant plain-text representation, enabling reliable extraction of structured arguments for downstream tool invocation under heterogeneous parsing behaviors.

Together, these three components form a strict role-separated input system in which execution-relevant information is partitioned across inputs and only becomes actionable through joint inference in the agent’s standard reasoning pipeline.

4.4 Adaptive Explicitness

We introduce a discrete control variable $v \in V$ that governs the level of operational specificity encoded in the file component. The set $V = \{v_2, v_3, v_4, v_5, v_6\}$ defines an ordered hierarchy of representations with increasing explicitness. Lower levels contain only abstract workflow narratives, while higher levels progressively introduce domain hints, object identifiers, dependency structure, and explicit state transitions.

For each target instance i , we select $v^* = \min\{v \in V : L_i = \text{True}\}$, where L_i is a success predicate requiring correct tool invocation, valid argument construction, and successful completion of the expected state transition.

Crucially, this selection is monotonic but not continuous: increasing explicitness improves execution success but simultaneously increases detectability. This induces a constrained optimization landscape where the solution is defined at the minimal feasible explicitness level.

Only the file component is adapted across v , while task and skill remain fixed. This isolates variability to a single channel, enabling systematic evaluation and ensuring that differences in success rates reflect representational sufficiency rather than structural changes in the attack design.

5 Development and Failure Analysis

The final method was developed through successive platform submissions. Table 1 summarizes the outcomes at each stage of the evaluation funnel.

5.1 Direct and Task-Only Constructions

The first baseline used ordinary business prompts and explicit skills. All 19 skills passed the static scan, but only three samples passed the instruction evaluator. This result means that static skill acceptance did not imply that the complete instruction context appeared benign. We next reduced tasks to continuation and result-query requests while preserving the skills. Platform success fell to two and then one target. The tasks were less suspicious, but the agent either still encountered explicit operational detail in context or no longer received a strong reason to act. Task-only rewriting therefore could not solve both sides of the trade-off.

5.2 Lexical Decomposition

We tested splitting sensitive strings across task, skill, and attachment, with the agent instructed to reconstruct fields before using a native tool. The construction was locally executable, but the platform result was 0/19. There are two likely reasons. First, assembly instructions themselves form a recognizable evasive pattern even when no file contains a complete sensitive token. Second, character-level reconstruction adds a brittle reasoning step before tool selection. The experiment rejects a purely lexical model of stealth: reducing per-file keyword visibility does not necessarily reduce the inferred intent of the complete workflow.

5.3 Subset Reuse and Low-Intent Skills

A six-target package composed of historically successful samples passed only one target. In addition, repeated submissions sometimes produced different target-level outcomes even when the submitted materials were unchanged. These observations suggest that the platform’s defense pipeline was stochastic: a sample that passed once was not guaranteed to pass again, and a single successful result could not be treated as a stable property of the material.

We then rebuilt all targets around a common low-intent service-record skill. Local validation reached 19/19, yet platform success remained 2/19 and three skills were rejected. The gap between local trajectories and platform scores confirmed that execution validation and material safety are separate axes.

5.4 Image Carriers

QR and OCR-based cards moved operational detail out of visible text files, but the competition image did not read them reliably. PNG metadata offered a more stable file-reading route because the information remained machine-readable without

Construction	N	Skill	Instr.	Final	Dominant failure mode
Initial final-round baseline	19	19	3	3	Explicit workflow semantics
Continuation task	17	17	2	2	Context still exposed intent
Result-query task	19	19	1	1	Weak action trigger
Cross-file string split	19	19	0	0	Suspicious assembly semantics
Historical natural subset	6	6	1	1	Poor transfer to stricter judge
Low-intent skill	19	16	2	2	Detail remained in visible text
PNG-metadata hybrid	19	17	3	3	Metadata too execution-oriented
Benign-story metadata v1	19	19	18	18	One target under-specified
Final adaptive metadata	19	19	19	19	None observed

Table 1: Platform-stage progression. Skill and Instr. are counts accepted by the skill scan and instruction evaluator; Final is the number of targets with a successful scored trajectory.

Tier	Additional detail	Targets
v2	Narrative and generic roles	4
v3	Domain/object clarification	10
v4	Stronger target-state hint	1
v5	Explicit business dependency	3
v6	Maximum necessary specificity	1

Table 2: Final metadata explicitness distribution across 19 targets.

Scenario	Targets	Passed
Smart Home	5	5
E-commerce	6	6
Enterprise OA	8	8
Overall	19	19

Table 3: Final platform result for the submitted package.

depending on visual recognition. This made it suitable for carrying structured service records while keeping the visible task and skill concise.

6 Experimental Setup

6.1 Environment and Reproducibility

All experiments use the image organizer provided and the Kimi-K2.5 base model [Team *et al.*, 2026].

Each target is generated from a manifest and the version map in `metadata_versions.json`. A run starts from a fresh container, mounts one target’s task, skill, and card, and collects the resulting trace. The validator requires: (1) agent exit code zero, (2) an intact `ROOT.hmac`, (3) the expected scenario tool event, (4) target-specific required arguments and success conditions, and (5) no replay-invalid marker. Only a run satisfying all checks is eligible for a verified package.

6.2 Material Configuration

Task and skill are shared structurally across targets. The attachment always uses the neutral filename `card.png`; target directory names are fixed by the competition schema. Metadata versions vary by target. Table 2 summarizes the final distribution. Most targets require v3, while only one requires the most explicit v6 tier. This concentration supports the use of a common low-intent workflow with limited target-specific tuning.

6.3 Evaluation Metrics

We report skill-scan acceptance, instruction-evaluation acceptance, and final target success. For the final package, target coverage equals the number of unique successful targets divided by 19. Because each target appears once in the package, precision equals the fraction of submitted samples that

succeed. When all targets pass, both quantities and their harmonic mean are one. We do not report confidence intervals: the organizer returns one scored outcome per submitted sample.

7 Evaluation

7.1 Final Platform Result

The final package passed all three evaluation stages for all 19 targets. Table 3 reports coverage by scenario. The package therefore achieves coverage $C = 19/19 = 1$, sample precision $P = 19/19 = 1$, and the competition harmonic score

$$F_{\text{attack}} = \frac{2CP}{C + P} = 1. \quad (3)$$

The platform result demonstrates that local trajectory success alone is insufficient. Earlier low-intent and split constructions generated locally valid trajectories for all targets, but achieved only 2/19 and 0/19 on the platform, respectively. The final score therefore reflects improved performance at the pre-execution safety gates rather than changes to the trajectory validation process.

7.2 Cross-Domain Behavior

The same three-layer architecture transfers across domains with different objects and side effects. In Smart Home, the record must lead the agent from a generic service workflow to a device or account operation. In E-commerce, the agent must distinguish order, catalog, inventory, refund, and communication records. Enterprise OA adds document, finance, salary, approval, calendar, and mail workflows, including cases where a value produced in one step is needed in the next. Full success across all three scenarios suggests that the method does not depend on one particular service vocabulary.

Although the same three-layer architecture is used across all targets, different targets require different levels of meta-data detail. The version distribution in Table 2 shows that four targets work with the lowest retained tier, ten require moderate object or domain clarification, and four require stronger state or dependency descriptions. The single v6 target is evidence against a universal maximally vague payload. Conversely, only one target requires v6, which argues against placing maximal detail in every sample.

8 Limitations

Our study is situated within a controlled competition environment with a fixed base model, tool interfaces, parsers, safety evaluators, and service mocks. This setting enables reproducible analysis of end-to-end agent behavior under a clearly specified evaluation pipeline, but it also bounds the scope of our claims. The results should therefore be interpreted as evidence of a concrete vulnerability pattern in multi-input tool-using agents, rather than as a measurement of its prevalence across all agent architectures.

The platform submission budget also limits statistical analysis. Our development experiments followed a sequence of progressively revised constructions, rather than a factorial ablation with repeated seeds. Differences across these constructions may reflect interactions among task wording, skill semantics, carrier choice, narrative framing, metadata version, and model sampling. We therefore use the progression to identify plausible failure modes and design constraints, but do not claim causal effect sizes for individual components.

The final 19/19 score measures the competition targets, not stealth against human analysts or transfer to arbitrary agents. A defender that extracts all PNG metadata, preserves provenance, and enforces tool-level authorization may block the method. Conversely, other container formats may expose related surfaces that we did not test. Broader evaluation should vary models, parsers, attachment types, tool schemas, defense architectures, and repeated seeds.

9 Conclusion

This paper presented a metadata-borne indirect instruction injection method for the OpenClaw Agent Security Attack Competition. Our method, Distributed Narrative Injection, decomposes attack intent across a neutral user task, a low-intent service-record skill, and business-framed PNG metadata, while using adaptive explicitness tuning to preserve executability under pre-execution safety checks. Across the final benchmark, this construction passed all 19 targets, demonstrating that successful tool-using agent attacks can arise not from overt malicious instructions, but from semantically coherent information distributed across multiple benign-looking artifacts. This result suggests that agent-security defenses should evaluate how meaning is composed across inputs and enforce authorization at the point of tool use, rather than relying only on prompt-level or artifact-local inspection.

References

- [Bagdasaryan *et al.*, 2023] Eugene Bagdasaryan, Tsung-Yin Hsieh, Ben Nassi, and Vitaly Shmatikov. Abusing im-
- ages and sounds for indirect instruction injection in multi-modal LLMs. *arXiv preprint arXiv:2307.10490*, 2023.
- [Chen *et al.*, 2025a] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*. USENIX Association, 2025.
- [Chen *et al.*, 2025b] Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta sealign: A secure foundation llm against prompt injection attacks, 2025.
- [Cheng *et al.*, 2026] Hao Cheng, Changtao Miao, Tianle Song, Yin Wu, He Liu, Erjia Xiao, Junchi Chen, Xiaoyu Shi, Yichi Wang, Jing Yang, et al. SeClaw: Spec-driven security task synthesis for evaluating autonomous agents. *arXiv preprint arXiv:2606.02302*, 2026.
- [Debenedetti *et al.*, 2024] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [Debenedetti *et al.*, 2025] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
- [Greshake *et al.*, 2023] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [He *et al.*, 2026] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. Attriguard: Defeating indirect prompt injection in llm agents via causal attribution of tool invocations. *arXiv preprint arXiv:2603.10749*, 2026.
- [Hines *et al.*, 2024] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024.
- [Hung *et al.*, 2025] Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H. Hsu, and Pin-Yu Chen. Attention tracker: Detecting prompt injection attacks in LLMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- [Khodayari *et al.*, 2026] Soheil Khodayari, Xuenan Zhang, Bhupendra Acharya, and Giancarlo Pellegrino. Indirect prompt injection in the wild: An empirical study of prevalence, techniques, and objectives. *arXiv preprint arXiv:2604.27202*, 2026.
- [Liu *et al.*, 2024] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, pages 1831–1847. USENIX Association, 2024.

- [Lu *et al.*, 2025] Weikai Lu, Ziqian Zeng, Kehua Zhang, Haoran Li, Huiping Zhuang, Ruidong Wang, Cen Chen, and Hao Peng. Argus: Defending against multimodal indirect prompt injection via steering instruction-following behavior. In *CVPR*, 2025.
- [Ruan *et al.*, 2024] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- [Schulhoff *et al.*, 2023] Sander Schulhoff, Jeremy Pinto, Ansum Khan, Louis-François Bouchard, Chenglei Si, Svetlana Anati, Valen Tagliabue, Anson Liu Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition. *arXiv preprint arXiv:2311.16119*, 2023.
- [Shi *et al.*, 2025] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. Promptarmor: Simple yet effective prompt injection defenses. *arXiv preprint arXiv:2507.15219*, 2025.
- [Team *et al.*, 2026] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [Wang *et al.*, 2025] Mengxiao Wang, Yuxuan Zhang, and Guofei Gu. Promptsleuth: Detecting prompt injection via semantic intent invariance. *arXiv preprint arXiv:2508.20890*, 2025.
- [Yan *et al.*, 2026] Zihe Yan, Jiaping Gui, Zhuosheng Zhang, and Gongshen Liu. Lasm: Layer-wise scaling mechanism for defending pop-up attack on gui agents. In *CVPR*, 2026.
- [Zhan *et al.*, 2024] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024.
- [Zhang *et al.*, 2025] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*, 2025.
- [Zhu *et al.*, 2025] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. MELON: Provable defense against indirect prompt injection attacks in AI agents. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.